SQL Practice Questions

Questions first, then answers - try before you peek

Nirmalya Mandal - SAP Labs Interview Prep

Study pack - part 7 of 8

Contents

How to use this

SQL is one of the highest-probability topics in an SAP technical interview - it sits at the meeting point of DBMS (your strongest core subject) and real querying. This document gives you **30 practice questions**, then **all the answers** in a separate section. Do it properly: read the schema, **write your own query on paper first**, then check the answer. Struggling for a minute before seeing the solution is what makes it stick.

The queries use standard SQL that works in PostgreSQL (what you actually use). A few notes:

- Interviewers care about **correct logic** over perfect dialect. If you write `LIMIT` and they use `TOP`, that's fine - explain what you mean.
- When they say "design a query," **think in this order:** which tables (FROM/JOIN), which rows (WHERE), any grouping (GROUP BY/HAVING), which columns (SELECT), and finally ordering/limiting (ORDER BY/LIMIT).
- The classic tricky ones - **second highest salary, per-group maximum, duplicates, above-average** - come up again and again. Master those.

The sample schema

All questions run against these tables.

Employees

Column	Type	Notes
emp_id	INT	primary key
name	VARCHAR	employee name
department_id	INT	foreign key to Departments
salary	INT	monthly salary
manager_id	INT	emp_id of their manager (can be NULL)
hire_date	DATE	date joined

Departments

Column	Type	Notes
department_id	INT	primary key
department_name	VARCHAR	e.g. Engineering
location	VARCHAR	city

Customers and **Orders** (for the join questions)

Customers	Orders
customer_id (PK)	order_id (PK)
name	customer_id (FK)
city	order_date
	amount

The questions

Write your query for each before looking at the answers section.

Basics (SELECT, WHERE, ORDER BY, DISTINCT)

1. Select all columns of all employees.
2. Select only the name and salary of every employee.
3. Find all employees in department 3.
4. Find all employees earning more than 50000.
5. List all employees sorted by salary, highest first.
6. List the distinct department_ids that appear in the Employees table.
7. Find employees whose name starts with the letter 'A'.
8. Find employees earning between 40000 and 60000 (inclusive).
9. Find employees who belong to department 1, 2, or 3 (use one operator).
10. Find employees who have no manager (manager_id is empty).

Aggregates and grouping (COUNT, SUM, AVG, GROUP BY, HAVING)

11. Count the total number of employees.
12. Find the average salary of all employees.
13. Find the highest and lowest salary in the company.
14. Find the total salary paid out by each department.
15. Find the number of employees in each department, ordered by count descending.
16. Find only the departments that have more than 5 employees.
17. Find the average salary per department, but only for departments whose average salary is above 50000.

Dates and patterns

18. Find all employees hired in the year 2023.
19. Find all customers whose city is 'Kolkata'.

Joins

20. Show each employee's name alongside their department_name.
21. Show all departments, including ones that currently have no employees.
22. Show each customer's name and their total order amount (customers with orders only).
23. Using a self-join, show each employee's name next to their manager's name.

The classic interview questions

24. Find the second highest salary in the Employees table.
25. Find the Nth highest salary (explain the general approach; show 3rd highest).
26. Find the highest-paid employee in each department.
27. Find employees who earn more than the overall average salary.
28. Find employees who earn more than the average salary of their own department.

29. Find duplicate employee names (names that appear more than once).
30. Find the top 3 highest-paid employees.

Bonus - modifying data and window functions

31. Give every employee in department 2 a 10% raise.
32. Delete all employees earning less than 20000.
33. Rank all employees by salary using a window function (highest = rank 1).

The answers

Compare your query to these. Small differences are fine if the logic is right.

Basics

1. All columns of all employees.

```sql
SELECT * FROM Employees;
```

2. Only name and salary.

```sql
SELECT name, salary FROM Employees;
```

3. Employees in department 3.

```sql
SELECT * FROM Employees
WHERE department_id = 3;
```

4. Employees earning more than 50000.

```sql
SELECT * FROM Employees
WHERE salary > 50000;
```

5. Sorted by salary, highest first.

```sql
SELECT * FROM Employees
ORDER BY salary DESC;
```

6. Distinct department_ids.

```sql
SELECT DISTINCT department_id FROM Employees;
```

7. Names starting with 'A'. The `%` means "any characters after."

```sql
SELECT * FROM Employees
WHERE name LIKE 'A%';
```

8. Salary between 40000 and 60000. `BETWEEN` is inclusive of both ends.

```sql
SELECT * FROM Employees
WHERE salary BETWEEN 40000 AND 60000;
```

9. Department 1, 2, or 3. `IN` is cleaner than three OR conditions.

```sql
SELECT * FROM Employees
WHERE department_id IN (1, 2, 3);
```

10. Employees with no manager. Use `IS NULL`, never `= NULL`.

```sql
SELECT * FROM Employees
WHERE manager_id IS NULL;
```

Aggregates and grouping

11. Total number of employees.

```sql
SELECT COUNT(*) FROM Employees;
```

12. Average salary.

```sql
SELECT AVG(salary) FROM Employees;
```

13. Highest and lowest salary.

```sql
SELECT MAX(salary) AS highest, MIN(salary) AS lowest
FROM Employees;
```

14. Total salary per department. Whenever you aggregate per group, `GROUP BY` that group.

```sql
SELECT department_id, SUM(salary) AS total_salary
FROM Employees
GROUP BY department_id;
```

15. Employee count per department, most first.

```sql
SELECT department_id, COUNT(*) AS num_employees
FROM Employees
GROUP BY department_id
ORDER BY num_employees DESC;
```

16. Departments with more than 5 employees. `HAVING` filters groups (after grouping); `WHERE` filters rows (before). That distinction is a very common interview question.

```sql
SELECT department_id, COUNT(*) AS num_employees
FROM Employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

17. Average salary per department, only where that average exceeds 50000.

```sql
SELECT department_id, AVG(salary) AS avg_salary
FROM Employees
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

Dates and patterns

18. Employees hired in 2023. Two clean ways:

```sql
SELECT * FROM Employees
WHERE EXTRACT(YEAR FROM hire_date) = 2023;

-- or, index-friendly:
SELECT * FROM Employees
WHERE hire_date >= '2023-01-01'
  AND hire_date <  '2024-01-01';
```

19. Customers in Kolkata.

```sql
SELECT * FROM Customers
WHERE city = 'Kolkata';
```

Joins

20. Employee name with department name. An INNER JOIN keeps only employees that have a matching department.

```sql
SELECT e.name, d.department_name
FROM Employees e
JOIN Departments d
  ON e.department_id = d.department_id;
```

21. All departments, even empty ones. A LEFT JOIN keeps every department; employees are NULL where none exist.

```sql
SELECT d.department_name, e.name
FROM Departments d
LEFT JOIN Employees e
  ON d.department_id = e.department_id;
```

22. Each customer's total order amount. Join, group by the customer, sum the amounts.

```sql
SELECT c.name, SUM(o.amount) AS total_amount
FROM Customers c
JOIN Orders o
  ON c.customer_id = o.customer_id
GROUP BY c.name;
```

23. Self-join: employee and their manager. Join the table to itself with two aliases. A LEFT JOIN keeps employees who have no manager.

```sql
SELECT e.name AS employee, m.name AS manager
FROM Employees e
```

```sql
LEFT JOIN Employees m
  ON e.manager_id = m.emp_id;
```

The classic interview questions

24. Second highest salary. The safest, most portable answer - find the max salary that is below the overall max:

```sql
SELECT MAX(salary) AS second_highest
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

Alternative with a window function (also good to mention):

```sql
SELECT DISTINCT salary
FROM Employees
ORDER BY salary DESC
OFFSET 1 LIMIT 1;
```

25. Nth highest salary (here, 3rd). `OFFSET N-1` skips the higher ones. `DISTINCT` avoids ties counting twice.

```sql
SELECT DISTINCT salary
FROM Employees
ORDER BY salary DESC
OFFSET 2 LIMIT 1;    -- 3rd highest: OFFSET = N-1
```

26. Highest-paid employee in each department. Cleanest with a correlated subquery:

```sql
SELECT * FROM Employees e
WHERE salary = (
    SELECT MAX(salary) FROM Employees
    WHERE department_id = e.department_id
);
```

Window-function version (mention this too - it handles ties clearly):

```sql
SELECT name, department_id, salary FROM (
    SELECT name, department_id, salary,
           RANK() OVER (PARTITION BY department_id
                        ORDER BY salary DESC) AS rnk
    FROM Employees
) t
WHERE rnk = 1;
```

27. Earn more than the overall average. A subquery computes the average once.

```sql
SELECT * FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

28. Earn more than their own department's average. A correlated subquery - the inner query runs per employee, scoped to their department.

```sql
SELECT * FROM Employees e
WHERE salary > (
    SELECT AVG(salary) FROM Employees
    WHERE department_id = e.department_id
);
```

29. Duplicate names. Group by the name and keep groups with more than one row.

```sql
SELECT name, COUNT(*) AS cnt
FROM Employees
GROUP BY name
HAVING COUNT(*) > 1;
```

30. Top 3 highest-paid employees.

```sql
SELECT * FROM Employees
ORDER BY salary DESC
LIMIT 3;
```

Bonus - modifying data and window functions

31. 10% raise for department 2. Always include the `WHERE`, or you update everyone.

```sql
UPDATE Employees
SET salary = salary * 1.10
WHERE department_id = 2;
```

32. Delete employees earning less than 20000.

```sql
DELETE FROM Employees
WHERE salary < 20000;
```

33. Rank all employees by salary. A window function computes a rank without collapsing rows (unlike GROUP BY).

```sql
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM Employees;
```

`RANK()` leaves gaps after ties (1,2,2,4); `DENSE_RANK()` does not (1,2,2,3); `ROW_NUMBER()` gives every row a unique number. Knowing the difference is a strong bonus point.

The five ideas that answer most SQL questions

If you internalise these, you can reason out almost any query on the spot:

1. **WHERE filters rows, HAVING filters groups.** WHERE runs before grouping, HAVING after.
2. **GROUP BY collapses rows into groups; window functions keep every row** while adding a computed column (rank, running total).
3. **A correlated subquery runs once per outer row** - it's how you do "compared to their own group" (per-department average, per-department max).
4. **JOIN type matters:** INNER keeps only matches; LEFT keeps all of the left table. "Include the ones with none" almost always means LEFT JOIN.
5. **Build queries in execution order:** FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT. Narrate this while you write - it shows structured thinking.